

WALRUS: An Efficient Decentralized Storage Network

George Danezis
Mysten Labs & University College
London (UCL)
London, UK
george@mystenlabs.com

Giacomo Giuliani
Mysten Labs
London, UK
giacomo@mystenlabs.com

Lefteris Kokoris Kogias
Mysten Labs
Athens, Greece
lefteris@mystenlabs.com

Markus Legner
Mysten Labs
Zurich, Switzerland
markus@mystenlabs.com

Jean-Pierre Smith
Mysten Labs
Zurich, Switzerland
jean-pierre@mystenlabs.com

Alberto Sonnino
Mysten Labs & University College
London (UCL)
London, UK
alberto@mystenlabs.com

Karl Wüst
Mysten Labs
Zurich, Switzerland
karl@mystenlabs.com

Abstract

Decentralized storage faces a fundamental trade-off between replication overhead, recovery efficiency, and security guarantees. Current approaches either rely on full replication, incurring substantial storage costs, or employ erasure-coding schemes that struggle with efficient recovery, especially under high churn. We present WALRUS, a decentralized blob storage system that addresses these limitations through multiple technical innovations.

At the core of WALRUS is RED STUFF, a two-dimensional erasure-coding protocol that achieves high security with only a 4.5× replication factor, while providing self-healing of lost data. This means that recovery is done without centralized coordination and requires bandwidth proportional to the amount of lost data.

However, RED STUFF on its own is not sufficient for WALRUS, as it is designed with a static set of participants in mind. To further support decentralization, we also introduce a multi-stage epoch-change protocol that efficiently handles storage node churn while maintaining uninterrupted availability during committee transitions. Our system incorporates authenticated data structures to defend against malicious clients and ensure data consistency throughout storage and retrieval. WALRUS has been deployed in production since March 2025 and has secured 686 TB of data by July 2026. We conduct an experimental evaluation of the deployed system and demonstrate that WALRUS achieves practical performance at scale.

CCS Concepts

• **Security and privacy** → **Distributed systems security**; *File system security*; • **Information systems** → **Distributed storage**; • **Computer systems organization** → **Dependable and fault-tolerant systems and networks**.



This work is licensed under a Creative Commons Attribution 4.0 International License.
CCS '26, The Hague, Netherlands

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2871-6/2026/11
<https://doi.org/10.1145/3830454.3832580>

Keywords

Decentralized storage; Byzantine fault tolerance; Blockchain; Erasure coding

ACM Reference Format:

George Danezis, Giacomo Giuliani, Lefteris Kokoris Kogias, Markus Legner, Jean-Pierre Smith, Alberto Sonnino, and Karl Wüst. 2026. WALRUS: An Efficient Decentralized Storage Network. In *Proceedings of the 2026 ACM SIGSAC Conference on Computer and Communications Security (CCS '26)*, November 15–19, 2026, The Hague, Netherlands. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3830454.3832580>

1 Introduction

Blockchains support decentralized computation through the State Machine Replication (SMR) paradigm [20]. However, they are practically limited to distributed applications that require little data for operation. Since SMR requires all validators to replicate data fully, it results in a large replication factor ranging from 100× to 1000×, depending on the number of validators in each blockchain.

While full data replication is needed for computing on state, it introduces substantial overhead when applications only need to store and retrieve binary large objects (blobs) without computing on them.¹ Dedicated decentralized storage networks [3] emerged to store blobs more efficiently. For example, early networks like IPFS [2] offer censorship resistance, reliability, and availability through replication [28].

Protocols for decentralized storage generally fall into two main categories. The first category includes systems using *replication*, with Filecoin [17] and Arweave [27] serving as prominent examples. The main advantage of these systems is the complete availability of the blob on selected storage nodes, which allows for easy access and seamless migration if a storage node goes offline. This setup enables a permissionless environment since storage nodes do not need to rely on each other for file recovery.

¹A recent example includes “inscriptions” on Bitcoin and other chains; see <https://medium.com/@thevalleylife/crypto-terms-explained-exploring-bitcoin-inscriptions-51699dc218d2>.

However, the reliability of these systems hinges on the robustness of the selected storage nodes. For instance, assuming a classic $1/3$ static adversary model and an unbounded pool of candidate nodes, achieving “twelve nines” of durability—a probability below 10^{-12} of losing access to a file—requires storing more than 25 copies on the network.² This results in a $25\times$ storage overhead. A further challenge arises from Sybil attacks [7], where malicious actors can pretend to store multiple copies of a file, undermining the system’s integrity.

The second category of decentralized storage services [10] uses *Reed-Solomon (RS) encoding* [19]. RS encoding reduces replication requirements significantly. For example, in a setting similar to that of a blockchain, with n nodes, of which $1/3$ may be malicious, and in an asynchronous network, RS encoding can achieve sufficient security with the equivalent of just $3\times$ storage overhead. This is possible since RS encoding splits a file into smaller pieces, which we call *slivers*, each representing a fraction of the original file. Any set of slivers greater in total size than the original file can be decoded back into the original file.

However, an issue with erasure coding arises when a storage node fails and needs to be replaced by another. Unlike fully replicated systems, where data can simply be copied from one node to another, RS-encoded systems require that all existing storage nodes send their slivers to the substitute node. The substitute can then recover the lost sliver, but this process results in $\mathcal{O}(|\text{blob}|)$ data being transmitted across the network. Frequent recoveries can erode the storage savings achieved through reduced replication. As a result, these systems need low churn among storage nodes, which is incompatible with permissionless settings.

In this work, we introduce WALRUS, a new approach to decentralized blob storage. It follows an erasure-coding architecture in order to scale to hundreds of storage nodes, providing high resilience at a low storage overhead. At the heart of WALRUS lies a new encoding protocol, called RED STUFF, which uses a novel two-dimensional (2D) encoding algorithm that is *self-healing*. Specifically, it enables the recovery of lost slivers using bandwidth proportional to the amount of lost data ($\mathcal{O}(|\text{blob}|/n)$ in our case). Moreover, RED STUFF incorporates authenticated data structures to defend against malicious clients, ensuring that the data remains consistent. Table 1 compares WALRUS with the two existing approaches.

One additional challenge for WALRUS, and in general for any encoding-based decentralized storage system, is operating securely across epochs, each managed by a different committee of storage nodes. This is challenging because we want to ensure uninterrupted availability to both read and write blobs during the naturally occurring churn of a permissionless system. Yet, if we keep writing data to the nodes about to depart, the departing nodes must keep transferring that data to the nodes replacing them. This creates a race for the resources of those nodes, which will either stop accepting writes or fail to ever transfer responsibility. WALRUS deals with this through its novel multi-stage epoch-change protocol that naturally fits the principles of decentralized storage systems.

In summary, we make the following contributions:

- We define the problem of Asynchronous Complete Data Storage and propose RED STUFF, the first protocol to solve it efficiently even under Byzantine faults (section 3).
- We present WALRUS, the first permissionless decentralized storage protocol designed for low replication cost and the ability to efficiently recover lost data due to faults or participant churn (section 4).
- We provide a production-grade implementation of WALRUS and deploy both a public testnet and a mainnet. We then measure its performance on testnet and report passive observations from mainnet (section 7).

2 Models and Definitions

2.1 Network and Adversarial Assumptions

WALRUS runs in epochs, each with a static set of storage nodes. At the end of the epoch, $n = 3f + 1$ storage nodes are elected as part of the storage committee of the next epoch and each one controls a storage *shard* such that a malicious adversary can control up to f of them.³ The corrupted nodes can deviate arbitrarily from the protocol. The remaining nodes are honest and adhere to the protocol. If a node controlled by the adversary at epoch e is not a part of the storage node set at epoch $e + 1$, then the adversary can adapt and compromise a different node at $e + 1$ after the epoch change completes.

We assume every pair of honest nodes has access to a reliable and authenticated channel. The network is asynchronous, so the adversary can arbitrarily delay or reorder messages between honest nodes, but must eventually deliver every message unless the epoch ends first. If the epoch ends, then the messages can be dropped.

2.2 Erasure Codes

As part of WALRUS, we propose a protocol for Asynchronous Complete Data Storage (ACDS) that uses a linear erasure-coding scheme. Although it is not necessary for the core parts of the protocol, we also assume, for some of our optimizations, that the encoding scheme is *systematic*, meaning that the source symbols of the encoding scheme also appear as part of its output symbols.

Let $\text{Encode}(B, t, n)$ be the encoding algorithm. Its output is n symbols such that any t can be used to reconstruct B . This happens by first splitting B into t symbols of size $\mathcal{O}(|B|/t)$ which are called *source* symbols. These are then extended by generating $n - t$ repair symbols for a total of n output symbols. On the decoding side, anyone can call $\text{Decode}(T, t, n)$ where T is a set of at least t correctly encoded symbols, and it returns the blob B .

ACDS shares some similarities with Asynchronous Verifiable Information Dispersal (AVID) [5, 15], given that the main goal of both protocols is to distribute data. However, they also have significant differences; most notably, AVID protocols do not provide completeness, which is critical for WALRUS. A more in-depth discussion is provided in section 8.

²The chance that all 25 storage nodes are adversarial and delete the file is $3^{-25} = 1.18 \times 10^{-12}$.

³For the protocol description, we identify each storage node with a single shard, which lets us state all thresholds over nodes. In general, the number of shards is a fixed protocol parameter and a node may control several of them, counting with the corresponding weight towards every threshold; sections 4.5 and 7 describe how shards are assigned to nodes and reassigned across epochs.

Table 1: Comparison of decentralized storage approaches.

	Replication for 10^{-12}	Write/Read Cost	Single Shard Recovery	Non-blocking Epoch Change
Replication [17, 27]	25×	$\mathcal{O}(n \text{blob})$	$\mathcal{O}(\text{blob})$	No
Classic ECC [21, 26]	3×	$\mathcal{O}(\text{blob})$	$\mathcal{O}(\text{blob})$	No
WALRUS + RED STUFF (this work)	4.5×	$\mathcal{O}(\text{blob})$	$\mathcal{O}(\text{blob} /n)$	Yes

2.3 Blockchain Substrate

WALRUS uses an external blockchain as a black box for all control operations that happen on WALRUS. A blockchain protocol can be abstracted as a computational black box that accepts a concurrent set of transactions, each with an input message $Tx(M)$ and outputs a total order of updates to be applied to the state $Res(seq, U)$. We assume that the blockchain does not deviate from this abstraction and does not censor $Tx(M)$ indefinitely. Any high-performance modern SMR protocol satisfies these requirements; in our implementation, we use Sui [4].

3 Asynchronous Complete Data Storage (ACDS)

We first define the problem of (Asynchronous) Complete Data Storage in a distributed system and describe our solution for an asynchronous network, which we call RED STUFF. We then show its correctness and complexity.

3.1 Problem Statement

In a nutshell, a Complete Data Storage protocol allows a writer to write a blob to a network of storage nodes (*Write Completeness*); ensures that readers read it consistently despite a potentially malicious writer (*Read Consistency*); and ensures that any reader can read it despite some failures and malicious behavior among storage nodes (*Validity*). More formally:

Definition 1 (Complete Data Storage). *Given a network of $n = 3f + 1$ nodes, where up to f are malicious, let B be a blob that a writer W wants to store within the network and share with a set of readers R . A protocol for Complete Data Storage guarantees three properties:*

- *Write Completeness: If a writer W is honest, then every honest node holding a commitment to blob B eventually holds a part p (derived from B), such that B can be recovered from $\mathcal{O}(|B|/|p|)$ parts.*
- *Read Consistency: Two honest readers, R_1 and R_2 , reading a successfully written blob B either both succeed and return B or both return $\perp$.*
- *Validity: If an honest writer W successfully writes B , then an honest reader R holding a commitment to B can successfully read B .*

Definition 2 (Asynchronous Complete Data Storage). *A Complete Data Storage protocol that is secure in asynchronous networks is an Asynchronous Complete Data Storage (ACDS) protocol.*

3.2 Strawman Design

In this section, we iterate through two strawman designs and discuss their inefficiencies.



Figure 1: Encoding a blob in one dimension. First, the blob is split into $f + 1$ systematic slivers and then a further $2f$ repair slivers are encoded.

Strawman I: Replication. The simplest protocol uses replication in the spirit of Filecoin [17] and Arweave [27]. The writer W broadcasts its blob B along with a binding commitment to B (e.g., $H_B = \text{hash}(B)$) to all storage nodes and then waits to receive $f + 1$ receipt acknowledgments. These acknowledgments form an availability certificate, which guarantees availability as at least one acknowledgment is from an honest node. The writer can publish this certificate on the blockchain, which ensures that it is visible to every other honest node, which can then successfully Read(B). This achieves Write Completeness since eventually all honest nodes will hold blob B locally. The rest of the properties also hold trivially. Note that the reader never reads $\perp$.

Although the full-replication protocol is simple, it requires the writer to send data of size $\mathcal{O}(n|B|)$ over the network, which is also the total cost of storage. Additionally, if the network is asynchronous, it can cost up to $f + 1$ requests to guarantee a correct replica is contacted, which would lead to $\mathcal{O}(n|B|)$ cost per recovering storage node with a total cost of $\mathcal{O}(n^2|B|)$ over the network. Similarly, even a read can be very inefficient in asynchrony, as the reader might need to send $f + 1$ requests with a cost of $\mathcal{O}(n|B|)$.

Strawman II: Encode & Share. To reduce the upfront data dissemination cost, some distributed storage protocols such as Storj [21] and Sia [26] use RS coding [19]. The writer W divides its blob B into $f + 1$ slivers and encodes $2f$ extra repair slivers (figure 1). Thanks to the encoding properties, any $f + 1$ slivers can be used to recover B . Each sliver has a size of $\mathcal{O}(|B|/n)$. The writer W then commits to the slivers using a binding commitment such as a Merkle tree [12] and sends each node a separate sliver together with a proof of inclusion. The nodes receive their slivers and check them against the commitment; if the sliver is correctly committed, they acknowledge by signing the commitment. The writer W can then generate an availability certificate from $2f + 1$ signatures and post it on the blockchain.

A reader continuously requests slivers from the nodes until it receives $f + 1$ valid replies (i.e., replies that are verified against the commitment). The reader is guaranteed to receive them since at least $f + 1$ honest nodes have stored their sliver. The reader then reconstructs blob B from the slivers and additionally re-encodes the recovered value and recomputes the commitment [6, 12]. If writer W was honest, the recomputed commitment will match the commitment from the availability certificate and the reader outputs B . Otherwise, writer W may not have committed to a valid encoding, in which case the commitments do not match and the reader outputs $\perp$.

As before, the nodes that did not get slivers during the sharing phase can recover them by reading B . If the output of the read operation is $\perp$, the node returns $\perp$ on all future reads. Otherwise, the node stores its encoded sliver and discards the rest of B . Note that this recovery process is expensive: recovery costs $\mathcal{O}(|B|)$ even if the storage cost afterwards is $\mathcal{O}(|B|/n)$.

This second protocol reduces the dissemination costs significantly at the expense of extra computation (encoding/decoding and committing to slivers from B). Disseminating blob B only costs $\mathcal{O}(|B|)$, which is the same cost as reading it. However, complete dispersal still costs $\mathcal{O}(n|B|)$, because, as we saw, the process of recovering missing slivers requires downloading the entire blob B . Given that there can be up to f storage nodes that did not manage to get their sliver from writer W and need to invoke the recovery protocol, the protocol has $\mathcal{O}(n|B|)$ total cost. This is not only important during the initial dispersal, but also in cases where the storage node set changes (at epoch boundaries) as the new set of storage nodes needs to read its slivers by recovering them from the previous set of storage nodes.

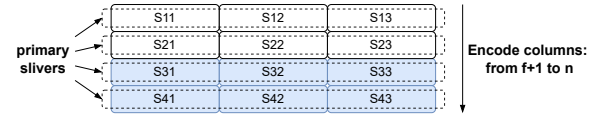
3.3 Final Design: RED STUFF

In this section we present the high-level design of RED STUFF. The algorithms can be found in section 5 and the proofs in section 6.

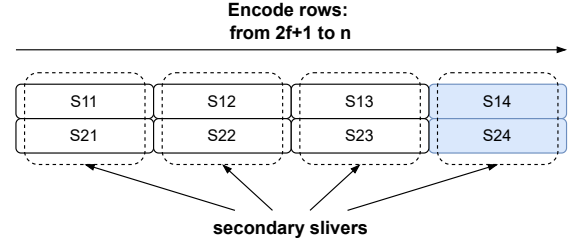
The encoding protocol above achieves the objective of a low overhead factor with very high assurance, but is still not suitable for a long-lasting deployment. The main challenge is that in a long-running large-scale system, storage nodes routinely experience faults, lose their slivers, and have to be replaced. Additionally, in a permissionless system, there is some natural churn of storage nodes even when they are well incentivized to participate.

Both of these cases would result in enormous amounts of data being transferred over the network, equal to the total size of data being stored, in order to recover the slivers for new storage nodes. This is prohibitively expensive. We instead want the system to be self-healing such that the cost of recovery is proportional only to the data that needs to be recovered and scales inversely with n .

RED STUFF achieves this. It encodes blobs in two dimensions (2D encoding): The primary dimension is equivalent to the RS encoding used in prior systems. However, in order to allow efficient recovery of slivers of B we also encode on a secondary dimension. RED STUFF is based on linear erasure coding (see section 2) and the Twin-code framework [18], which provides erasure-coded storage with efficient recovery in a crash-tolerant setting with trusted writers.



(a) Primary encoding in two dimensions. The blob is split into $2f + 1$ columns and $f + 1$ rows. Each column is encoded as a separate blob with $2f$ repair symbols. Then each resulting row is the primary sliver of the respective node.



(b) Secondary encoding in two dimensions. The blob is split into $2f + 1$ columns and $f + 1$ rows. Each row is encoded as a separate blob with f repair symbols. Then each resulting column is the secondary sliver of the respective node.

Figure 2: 2D encoding / RED STUFF.

We adapt this framework to make it suitable in the Byzantine fault-tolerant (BFT) setting with a single set of storage nodes, and we add additional optimizations that we describe further below.

Encoding. Our starting point is the second strawman that splits the blob into $f + 1$ slivers. Instead of simply encoding repair slivers, we first add one more dimension to the splitting process: the original blob is split into $f + 1$ primary slivers (horizontal in the figure) and into $2f + 1$ secondary slivers (vertical in the figure). Figure 2 illustrates this process. As a result, the blob is now split into $(f + 1)(2f + 1)$ symbols that can be visualized in an $[f + 1, 2f + 1]$ matrix.

Given this matrix we then generate repair symbols in both dimensions. We take each of the $2f + 1$ columns (of size $f + 1$) and extend them to n symbols such that there are n rows. We assign each of the rows as the *primary sliver* of a node (figure 2a). This almost triples the total amount of data we need to send and is very close to what 1D encoding does in Strawman II. In order to provide efficient recovery for each sliver, we also take the initial $[f + 1, 2f + 1]$ matrix and extend each of the $f + 1$ rows (of size $2f + 1$) with repair symbols to n symbols (figure 2b) using our encoding scheme. This creates n columns, which we assign as the *secondary sliver* of each node, respectively.

For each sliver, we compute a vector commitment over all symbols of the extended sliver (i.e., including the repair symbols), forming the *sliver commitment*. The commitments for every sliver form the blob *metadata*. Using these, nodes can later, when queried for a single symbol, prove that the symbol they return is the symbol originally written, which enables Byzantine fault tolerance for RED STUFF. A vector commitment to the metadata then forms the *blob commitment*.

Write Protocol. The Write protocol of RED STUFF uses a pattern similar to that of the RS-code protocol. The writer W first encodes the blob and creates a sliver pair for each node. A sliver pair i is the pair of the i^{th} primary and secondary slivers. There are $n = 3f + 1$ sliver pairs, one per node.

Then, W sends the metadata (i.e., all sliver commitments) to every node, along with the respective sliver pair. The nodes check their own sliver pair against the commitments, recompute the blob commitment, and reply with a signed acknowledgment if all checks pass. Once $2f + 1$ signatures are collected, W aggregates them into a certificate and posts it on-chain to *certify* the blob's availability. The nodes then keep openings of the blob commitment to their sliver commitments, locally encode the metadata with a 1D $(f + 1)$ -out-of- n encoding (along with another vector commitment to the parts of the encoded metadata), and keep the part assigned to them, together with the opening of their part. This is needed since the metadata needs to be available for nodes that need to recover their slivers (see *Sliver Healing* below). However, fully replicating the metadata would result in a quadratic overhead, while this approach reduces the overhead to a constant per node (i.e., from quadratic to linear system-wide overhead). Our production deployment currently forgoes this optimization and stores the metadata in full on every node.

In theoretical asynchronous network models with reliable delivery, the above would result in all correct nodes eventually receiving a sliver pair from an honest writer. However, in practical protocols, the writer needs to stop retransmitting. It is safe to stop the retransmission after $2f + 1$ signatures are collected, leading to at least $f + 1$ correct nodes (out of the $2f + 1$ that responded) holding a sliver pair for the blob.

Read Protocol. The Read protocol is similar to the one for one-dimensional RS codes. It starts with R collecting the metadata, i.e., the list of sliver commitments for the blob commitment. To do so, R requests the 1D encoded metadata parts from the nodes along with the opening proofs for the encoded metadata. Since at least $f + 1$ of any $2f + 1$ replies originate from honest nodes, the vector commitment on which the majority of the replies agree is the correct one; parts whose openings do not verify against it are discarded before decoding.

Since honest nodes only store metadata that they verified against id when it was written, this binds the decoded metadata to the blob. Then R requests a read for the blob commitment from all nodes and they respond with the primary sliver they hold (this may happen gradually to save bandwidth). Each response is checked against the corresponding commitments in the commitment set for the blob. When $f + 1$ correct primary slivers are collected, R decodes B , re-encodes it, and recomputes the metadata. If it matches the metadata retrieved earlier, then R outputs B ; otherwise, it outputs $\perp$.

Sliver Healing. The big advantage of RED STUFF compared to the RS-code protocol is its self-healing property. This comes into play when nodes that did not receive their slivers directly from W try to recover them. Any storage node can recover its secondary sliver by asking $f + 1$ storage nodes for the symbols that exist in their row, which should also exist in the (extended) column of the requesting node (figures 3a to 3c). This means that eventually all $2f + 1$ honest nodes will have secondary slivers. At that point, any node can also

recover its primary sliver by asking the $2f + 1$ honest nodes for the symbols in their column (figure 3d) that should also exist in the (extended) row of the requesting storage node. In each case, the responding node also sends the opening for the requested symbol of the commitment of the source sliver. This allows the receiving node to verify that it received the symbol intended by the writer W , which ensures correct decoding if W was honest.

Since the size of a symbol is $\mathcal{O}(|B|/n^2)$, and each storage node will download $\mathcal{O}(n)$ total symbols, the cost per node remains at $\mathcal{O}(|B|/n)$ and the total cost to recover the blob is $\mathcal{O}(|B|)$, which is equivalent to the cost of a Read and of a Write. As a result, by using RED STUFF, the communication complexity of the protocol is (almost⁴) independent of n , making the protocol scalable.

Security. Section 6 provides proofs that RED STUFF satisfies all properties of an ACDS. Informally, Write Completeness is ensured by the fact that a correct writer will confirm that at least $f + 1$ correct nodes received sliver pairs before stopping retransmissions. The sliver recovery algorithm can then ensure that the remaining honest nodes can efficiently recover their slivers from these, until all honest nodes eventually hold their respective sliver, or can prove that the encoding was incorrect. Validity holds due to the fact that $2f + 1$ correct nodes will eventually hold correct sliver pairs, and therefore a reader that contacts all nodes will eventually get enough slivers to recover the blob. Read Consistency holds since two correct readers that decode a blob from potentially different sets of slivers, re-encode it and check the correctness of the encoding. Either both output the same blob if it was correctly encoded or both output $\perp$ if it was incorrectly encoded.

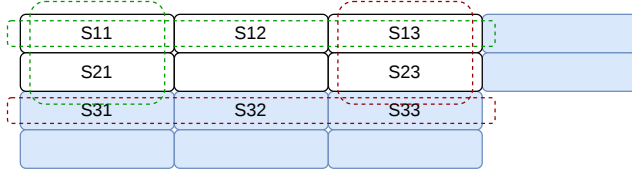
Discussion on Replication Cost. The benefits of RED STUFF do not come for free. There is an increase in the total storage cost of the system compared to classic BFT erasure codes, from $3\times$ to $4.5\times$. But this is necessary, as we need the extra information to allow for efficient recovery. Without it, the recovery cost for a single sliver would be $\mathcal{O}(|B|)$ instead of $\mathcal{O}(|B|/n)$.

A naive application of our 2D encoding in which nodes store fully extended slivers would lead to a storage amplification of $9\times$. However, nodes only store a sufficient number of symbols ($2f + 1$ for primary, $f + 1$ for secondary) that allows them to locally decode the rest of the extended sliver when they need to communicate symbols that are not stored to support another node's recovery, resulting in a factor $4.5\times$ instead. This trades CPU for storage, which is a favorable trade-off for WALRUS as it has minimal CPU requirements. Note additionally that we do not use a symmetric $[f + 1, f + 1]$ matrix, as it would result in double the replication cost. Instead, our design achieves the minimum replication cost at the expense of a slightly more structured recovery procedure.

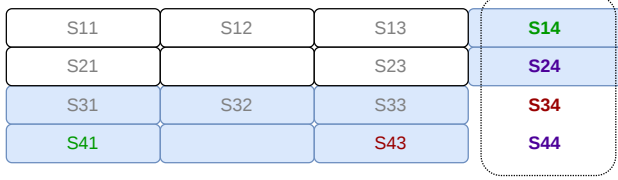
4 WALRUS: A Decentralized Blob Store

The previous section presented RED STUFF, an encoding algorithm designed for efficient recovery. However, an encoding algorithm alone is not sufficient for a complete storage system. In this section, we present WALRUS, which integrates an encoding algorithm with

⁴Depends on the vector commitment scheme used.



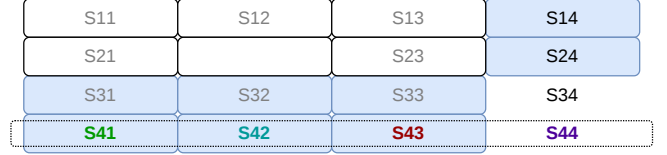
(a) Nodes 1 (green) and 3 (red) hold a primary and secondary sliver each, corresponding to an unextended row and column, respectively.



(c) Node 4 uses the $f + 1$ symbols on its column to recover the missing symbols of its extended secondary sliver (purple). It then sends any other recovering node the recovered intersections of this column with their row.



(b) Nodes 1 and 3 extend their slivers and send the intersection of the resulting rows/columns with the columns/rows of node 4 to node 4 (bold green and red).



(d) Node 4 uses the $f + 1$ symbols on its row that it received from nodes 1 and 3, symbol S44, which it recovered in the previous step (purple), and at least $f - 1$ further symbols sent by other nodes after they recover their own secondary slivers (not needed in this example) to recover the missing symbols of its primary sliver (teal).

Figure 3: Nodes 1 and 3 help node 4 recover its sliver pair.

a blockchain that serves as a control plane for metadata and governance. Within an epoch, clients interact with WALRUS through two operations: writing blobs and reading blobs.

WALRUS requires an Asynchronous Complete Data Storage (ACDS) protocol and a blockchain to coordinate storage nodes and track metadata. Our implementation uses RED STUFF (section 3.3) instantiated with Reed-Solomon codes [19] for the erasure coding, Merkle trees [12] for the vector commitments, and the Sui blockchain [4] as the control plane. These choices are not fundamental; WALRUS can use any components that satisfy the requirements in section 2.

Beyond basic read and write operations, WALRUS must also handle an additional concern: storage nodes may join or leave the system over time. Section 4.5 presents our epoch-change algorithm, which enables permissionless participation of hundreds of nodes with seamless handover between committees.

4.1 Writing a Blob

The process of writing a blob in WALRUS can be seen in algorithm 3 and figure 4. The process begins with the writer (1) encoding a blob using RED STUFF. This process yields sliver pairs, a list of sliver commitments, and a blob commitment. The writer derives a *blob ID* by hashing the blob commitment with additional metadata such as the length of the file and the type of the encoding (algorithm 1).

Then, the writer (2) submits a transaction on the blockchain to acquire sufficient space for the blob to be stored during a sequence of epochs, and to *register* the blob. The blob ID and the blob size are sent. The blockchain smart contract needs to secure sufficient space to store both the encoded slivers on each node and all metadata associated with the commitments for the blob. Some payment may be sent along with the transaction to secure empty space, or empty

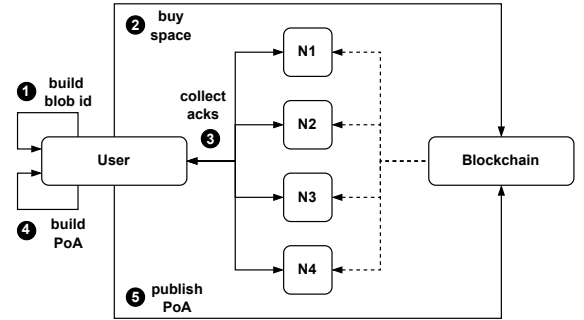


Figure 4: WALRUS write flow. The writer generates the blob ID of the file to store, acquires storage space through the blockchain, submits the encoded file to WALRUS, collects $2f+1$ acknowledgments, and submits them as proof of availability to the blockchain.

space over epochs can be a resource that is attached to this request to be used. Our implementation allows for both options.

Once the register transaction commits (5), the writer informs the storage nodes of their obligation to store the slivers of the blob identified by *id*, sending them the transaction together with the commitments and the primary and secondary slivers assigned to the respective storage nodes along with proofs that the slivers are consistent with the published *id*. This proof is the list of sliver commitments itself: the storage node recomputes *id* from the commitments and checks that it matches the registered blob ID; it then recomputes the commitment of each of the two slivers it received

and compares the results against the corresponding entries (algorithm 1). After the sliver pairs and commitments are verified and stored, the node responds with a signed acknowledgment over id .

Finally, the writer waits to collect $2f + 1$ signed acknowledgments (4), which constitute an availability certificate. This certificate is then published on-chain (5), which denotes the *Point of Availability* (PoA) for the blob in WALRUS. The PoA signals the obligation for storage nodes to keep the slivers available for reads. At this point, the writer can delete the blob from local storage and go offline. Additionally, this PoA can be used as proof of availability of the blob by the writer to third-party users and smart contracts.

Nodes listen to the blockchain for events indicating that a blob reached its PoA. If they do not hold sliver pairs for this blob, they execute the recovery process to get commitments and sliver pairs for all blobs past their PoA. This ensures that eventually all correct nodes will hold sliver pairs for all blobs.

4.2 Reading a Blob

Reading a blob proceeds in three phases: metadata retrieval, sliver collection, and reconstruction with verification. Given a blob ID id , the client first requests the metadata from the storage nodes. Once it has retrieved the metadata, the client proceeds to request primary slivers. The client collects primary slivers from storage nodes, verifying each against the sliver commitments from the metadata. After obtaining $f + 1$ valid slivers, the client reconstructs the blob using erasure decoding.

The final verification step ensures consistency: the client re-encodes the reconstructed blob and recomputes the metadata. If this matches the metadata retrieved earlier, the client outputs the blob; otherwise, the encoding was inconsistent and the client outputs $\perp$. This check guarantees that all honest readers either obtain the same blob or reject it entirely—a property that follows directly from RED STUFF’s guarantees. In the fault-free case, the total download is only slightly larger than the original blob size, since primary slivers contain the blob data with modest redundancy overhead. Moreover, in a steady state, reads are still available with up to $2f$ faulty shards, as only $f + 1$ slivers are needed.

4.3 Recovery of Slivers

One issue with writing blobs in asynchronous networks or when nodes crash-recover is that not every node gets their sliver during the write. This is not a problem, as protocols can function without completeness. Nevertheless, in WALRUS we opted to use a two-dimensional encoding scheme because it allows for completeness, i.e., the ability of every honest storage node to recover and eventually hold a sliver for every blob past PoA. This allows (1) better load balancing of read requests as all nodes can reply to reads and (2) dynamic availability of nodes, which enables reconfiguration without needing to reconstruct and rewrite every blob.

All these benefits rely on the ability of storage nodes to recover their slivers efficiently. The protocol closely follows the RED STUFF recovery protocols illustrated in figure 3. When a storage node sees a certificate of a blob for which it has not received slivers, it tries to recover its sliver pair from the rest of the storage nodes. For this, it requests from all storage nodes the symbols corresponding to the intersection of the recovering node’s primary/secondary sliver with

the signatory nodes’ secondary/primary slivers. Given that $2f + 1$ nodes signed the certificate, at least $f + 1$ will be honest and reply. This is sufficient for all $2f + 1$ honest nodes to eventually hold their secondary slivers. As a result, when all honest nodes hold their secondary slivers, they can share those symbols corresponding to the recovering nodes’ primary slivers; those nodes then reach the $2f + 1$ threshold and also recover their primary slivers.

4.4 Handling Byzantine Faults during Encoding

One last challenge for WALRUS is handling Byzantine faults, both for writers and for storage nodes. First, storage nodes could misbehave by acknowledging slivers that they do not hold or by serving wrong slivers during read/recovery. We defend against these attacks through the commitment to slivers that the writer creates, meaning that any invalid sliver will be rejected (algorithm 1, line 28).

Second, a malicious client could upload slivers that are inconsistent, i.e., do not correspond to a correct encoding of a blob. In that case, a node may not be able to recover a sliver that is consistent with the commitment of the symbols that it received. WALRUS does not require certified blobs to be recoverable (which is the primary goal of AVID [5]), as the writer can always encode recoverable garbage data. Instead, it is guaranteed that unrecoverable blobs will have a third-party-verifiable proof of inconsistency, associated with id , after a read fails.

The read process executed by a correct reader rejects any inconsistently encoded blob by default (algorithm 1, line 35), and as a result, sharing this proof is not a necessity to ensure consistent reads. However, agreeing on the inconsistency allows nodes to delete this blob’s data and thus save space and bandwidth.

To prove inconsistency, the storage node shares the inconsistency proof—consisting of the symbols that it received for recovery and their inclusion proofs—with the other nodes, who can verify it by performing a trial recovery themselves. After verifying this fraud proof, each verifying node attests on-chain that id is invalid. After observing $f + 1$ such attestations, all nodes will subsequently respond with $\perp$ to any request for the inconsistent blob’s slivers, along with a pointer to the on-chain evidence for the inconsistency.

This means that even if a malicious writer colludes with f malicious nodes to certify an invalid blob, the rest of the storage nodes will detect this during a read or recovery and produce an inconsistency proof, guaranteeing agreement among honest readers.

4.5 Committee Reconfiguration

The number of shards is a protocol parameter, fixed at 1000 in the current deployment, and $n = 3f + 1$ counts shards rather than operators. The number of shards therefore does not grow with the number of nodes in the committee in practice: shards are assigned to nodes in proportion to their delegated stake, so a larger committee only changes how the same shards are distributed and never requires re-encoding. The security invariant is stated over shards as well, requiring that the adversary control at most $1/3$ of them. WALRUS is orthogonal to the mechanism that enforces or incentivizes this invariant: the current deployment uses delegated proof of stake, but proof of work, proof of personhood, or random committee sampling from a larger population would serve equally well.

WALRUS is a decentralized protocol; therefore, it is natural that the set of storage nodes will fluctuate between epochs. When a new committee replaces the current committee between epochs, reconfiguration takes place. The goal of the reconfiguration protocol is to preserve the invariant that all blobs past the PoA are available, regardless of whether the set of storage nodes changes. Additionally, WALRUS must continue to perform reads and writes for blobs to ensure no downtime even if the reconfiguration process takes hours.⁵

Core Design. At a high level, the reconfiguration protocol of WALRUS is similar to the reconfiguration of blockchain systems, since WALRUS also operates in quorums of storage nodes. However, the reconfiguration of WALRUS has its own challenges because the migration of state is orders of magnitude more expensive than in classic blockchain systems. The most important challenge is the race between writing blobs for epoch e and transferring slivers from outgoing storage nodes to incoming storage nodes during the reconfiguration event between e and $e + 1$. More specifically, if the amount of data written in epoch e exceeds what the departing storage node can transfer to the incoming storage node, then the epoch will never finish. This problem is exacerbated when some of the outgoing storage nodes of e are unavailable, as this means that the incoming storage nodes need to recover the slivers from the committee.

To resolve this problem without shutting off the write path, we take a different approach than blockchain protocols. Specifically, we require writes to be directed to the committee of $e + 1$ the moment the reconfiguration starts, while still directing reads to the old committee, instead of having a single point at which both reads and writes are handed over to the new committee. We can afford to do this, unlike blockchains, because WALRUS does not handle its state consistency (which is the responsibility of Sui in our deployment) and hence can have divergence and duplication. However, this creates challenges when it comes to reading fresh blobs, as during the handover period it is unclear which nodes store the data. We handle this by including in the metadata of every blob the epoch in which it was first written. If the epoch is $e + 1$, then the client is asked to direct reads to the new committee; otherwise, it can direct reads to the old committee. This happens only during the handover period (when both committees need to be live and secure).

Once a member of the new committee has bootstrapped its part of the state, i.e., it has received all slivers for its shards, it signals that it is ready to take over. When nodes of the new committee collectively holding $2f + 1$ shards have signaled this, the reconfiguration process terminates and all reads are redirected to the storage nodes of the new committee.

The Importance of RED STUFF for Reconfiguration. As we mentioned above, a significant challenge arises during epoch change when some nodes are faulty. In prior work, a single faulty node would require bandwidth equal to the size of the file to be transferred over the network of storage nodes. This makes epoch changes prohibitively expensive and is the reason no prior decentralized system has one. The key enabler of WALRUS's ability to handle this gracefully is our RED STUFF algorithm, as it allows for the

bandwidth cost of the faulty case to be on the same order as that of the fault-free case. We discuss the associated overheads in section 7.

Security Arguments. In a nutshell, reconfiguration ensures all ACDS properties across epochs. The key invariant is that the reconfiguration algorithm ensures that if a blob is to be available across epochs, in each epoch $f + 1$ correct shards (potentially different ones) hold slivers. This is the purpose of the explicit signaling, by nodes collectively holding $2f + 1$ shards, that unlocks the epoch change. Therefore, eventually all other honest storage nodes can recover their sliver pairs, and in all cases, honest nodes in the next epoch are able to recover correct sliver pairs for $f + 1$ shards as a condition to move epochs.

5 Detailed Algorithms

This section supplements section 4 by providing detailed algorithms for client and storage node operations (algorithms 2 and 3).

In addition to the helper functions specified in algorithm 1, these algorithms also leverage the following (intuitive) functions: `BYTESIZE( $B$ )` to compute the size of a blob B in bytes; `ERASURE-DECODE( $T, t, n$ )` to erasure-decode the source data from a set T of at least t out of n correctly encoded symbols or slivers (section 2); `ERASURE-ENCODE( $D, t, n$ )` to erasure-encode data D into n symbols such that any t of them can recover D (section 2); `HASH( $\cdot$ )` to compute a cryptographic hash; `INCONSISTENCYPROOF( $T$ )` to assemble from the collected symbols and openings T a third-party-verifiable proof that the encoding is inconsistent (section 4.4); `MAJORITY( $v$ )` to return the most frequent element of a vector v ; `MERKLETREE( $v$ )` to compute a Merkle tree over a vector input v ; `NODEINDEX( $N, nodes$ )` to obtain the index of node N in the committee; `OPENING( $v, i$ )` to compute the opening for the entry at index i of the vector commitment to v ; `REQUESTSYMBOL( $m, id, (r, c)$ )` to request the symbol at position (r, c) of the blob id from node m , together with an opening against the commitment of the sliver that contains it; `SEND( $N, msg$ )` to send the message msg to node N over the authenticated channel; `SIGN( $x$ )` to sign x with the node's key; `SPLITINTOMATRIX( $D, r \times c$ )` to reshape data D into a matrix of the specified size; and `VERIFY-OPENING( $\tau, x, \pi, i$ )` to verify with the opening π that x is the entry at index i of the vector committed to by τ .

Furthermore, the client and storage nodes use the following functions to interact with the blockchain: `ISREGISTERED( $id$ )` to check if a blob ID id is registered on the blockchain; `READCERTIFICATE( $id$ )` to read an availability certificate for blob ID id from the blockchain; `RESERVEBLOB( $id, size, expiry$ )` to acquire storage for a blob of size $size$ until epoch $expiry$ and register the blob ID id on the blockchain; and `STORECERTIFICATE( $C_A, id$ )` to store the availability certificate C_A for blob ID id on the blockchain.

Table 2 summarizes the main notations used in the algorithms. Subscripts of matrices and vectors denote access to a specific index.

6 RED STUFF Proofs

This section formally proves that RED STUFF satisfies all properties of an ACDS scheme.

6.1 Write Completeness

We show that RED STUFF satisfies Write Completeness. Informally, if an honest writer writes a blob B to the network, every honest

⁵Unlike blockchain protocols where reconfiguration simply transfers responsibility for validation, in WALRUS it transfers responsibility for data storage on the order of terabytes or even petabytes.

Algorithm 1 WALRUS helper functions.

```

1: nodes  $\triangleright$  committee of storage nodes,  $|\text{nodes}| = n = 3f + 1$ 

2: procedure ENCODEBLOB( $B$ )
3:    $E \leftarrow \text{SPLITINTOMATRIX}(B, (f + 1) \times (2f + 1)) \triangleright$  the source symbols
4:    $E \leftarrow [\text{ERASUREENCODE}(E_{(*,j)}, f + 1, n) : j \in [0, 2f + 1]]^\top$ 
5:    $E \leftarrow [\text{ERASUREENCODE}(E_{(i,*)}, 2f + 1, n) : i \in [0, n]]$ 
6:    $S^p \leftarrow [E_{(i,*)} : i \in [0, n]] \triangleright$  primary slivers, one per node
7:    $S^s \leftarrow [E_{(*,j)} : j \in [0, n]]^\top \triangleright$  secondary slivers, one per node
8:   return  $(S^p, S^s)$ 

9: procedure MAKEMETADATA( $S^p, S^s$ )
10:   $M^p \leftarrow [\text{MERKLETREE}(S^{(p,i)}) : i \in [0, n]]$ 
11:   $M^s \leftarrow [\text{MERKLETREE}(S^{(s,i)}) : i \in [0, n]]$ 
12:   $M \leftarrow (M^p, M^s)$ 
13:  return  $M$ 

14: procedure MAKEBLOBID( $M, \text{size}$ )
15:   $(M^p, M^s) \leftarrow M$ 
16:   $(h^p, h^s) \leftarrow (\text{MERKLETREE}(M^p), \text{MERKLETREE}(M^s))$ 
17:   $h \leftarrow \text{HASH}(h^p, h^s) \triangleright$  the blob commitment
18:   $id \leftarrow \text{HASH}(h, \text{size})$ 
19:  return  $id$ 

20: procedure ENCODEMETADATA( $M$ )
21:   $m \leftarrow \text{ERASUREENCODE}(M, f + 1, n) \triangleright n$  metadata shares
22:   $\tau \leftarrow \text{MERKLETREE}(m)$ 
23:  return  $[(i, m_i, \tau, \text{OPENING}(m, i)) : i \in [0, n]] \triangleright$  one piece per node

24: procedure DECODEMETADATA( $P$ )  $\triangleright$  metadata pieces  $(i, m, \tau, \pi)$ 
25:   $\tau^* \leftarrow \text{MAJORITY}([\tau : (i, m, \tau, \pi) \in P]) \triangleright$  by honest majority
26:   $V \leftarrow \{(i, m) : (i, m, \tau, \pi) \in P, \text{VERIFYOPENING}(\tau^*, m, \pi, i)\}$ 
27:  return  $\text{ERASUREDECODE}(V, f + 1, n) \triangleright$  verified shares

28: procedure VERIFYSLIVER( $S^{(x,i)}, M$ )  $\triangleright$  sliver type  $x \in \{p, s\}$ 
29:   $(M^p, M^s) \leftarrow M$ 
30:  return  $\text{MERKLETREE}(S^{(x,i)}) = M_i^x$ 

31: procedure DECODEBLOBFROMPRIMARY( $\{S^{(p,*)}\}_{f+1}, M$ )
32:   $B \leftarrow \text{ERASUREDECODE}(\{S^{(p,*)}\}_{f+1}, f + 1, n)$ 
33:   $(S^p, S^s) \leftarrow \text{ENCODEBLOB}(B) \triangleright$  re-encode
34:   $M' \leftarrow \text{MAKEMETADATA}(S^p, S^s)$ 
35:  if  $M' \neq M$  then return  $\perp$ 
36:  return  $B$ 

```

storage node eventually holds a correctly encoded primary and secondary sliver of B . For this part we assume the writer is honest and provides a correct vector commitment M .

Lemma 1 (Primary Sliver Reconstruction). *If a party holds a set of $(2f + 1)$ symbols $\{E_{(i,*)}\}_{2f+1}$ from a primary sliver $S^{(p,i)}$, it can obtain the complete primary sliver $S^{(p,i)}$.*

PROOF. The proof directly follows from the reconstruction property of erasure codes with reconstruction threshold $(2f + 1)$. $\square$

Algorithm 2 WALRUS client operations.

```

1: nodes

// Store a blob
2: procedure STOREBLOB( $B, \text{expiry}$ )
3:   $(S^p, S^s) \leftarrow \text{ENCODEBLOB}(B)$ 
4:   $M \leftarrow \text{MAKEMETADATA}(S^p, S^s)$ 
5:   $\text{size} \leftarrow \text{BYTESIZE}(B)$ 
6:   $id \leftarrow \text{MAKEBLOBID}(M, \text{size})$ 
7:   $\text{RESERVEBLOB}(id, \text{size}, \text{expiry})$ 
8:   $R \leftarrow \{\}$ 
9:  for  $i \in [0, |\text{nodes}|]$  do
10:    $N \leftarrow \text{nodes}[i]$ 
11:    $\text{StoreRqst} \leftarrow (id, \text{size}, M, S^{(p,i)}, S^{(s,i)})$ 
12:    $R \leftarrow R \cup \{(N, \text{StoreRqst})\}$ 
13:   $C_A \leftarrow \text{await}_{2f+1} : \{c \leftarrow \text{SEND}(N, r) : (N, r) \in R\} \triangleright$  signed acks
14:   $\text{STORECERTIFICATE}(C_A, id) \triangleright$  the availability certificate

// Retrieve metadata
15: procedure RETRIEVEMETADATA( $id$ )
16:   $\text{MetadataRqst} \leftarrow (id)$ 
17:   $P_M \leftarrow \text{await}_{2f+1} : \{p \leftarrow \text{SEND}(N, \text{MetadataRqst}) : N \in \text{nodes}\} \triangleright$  metadata pieces
18:  return  $\text{DECODEMETADATA}(P_M)$ 

// Read from primary
19: procedure READBLOB( $id$ )
20:   $M \leftarrow \text{RETRIEVEMETADATA}(id)$ 
21:   $\text{SliversRqst} \leftarrow (id)$ 
22:   $\{S^{(p,*)}\}_{f+1} \leftarrow \text{await}_{f+1} : \{S^{(p,i)} \leftarrow \text{SEND}(N, \text{SliversRqst}) : N = \text{nodes}[i], \text{VERIFYSLIVER}(S^{(p,i)}, M)\}$ 
23:   $B \leftarrow \text{DECODEBLOBFROMPRIMARY}(\{S^{(p,*)}\}_{f+1}, M)$ 
24:  return  $B$ 

```

Table 2: Main notations used in the WALRUS algorithms.

$E_{(i,j)}$	Symbol at position (i, j) of an encoded blob
S^p	The set of primary slivers
S^s	The set of secondary slivers
$S^{(p,i)}$	The primary sliver held by storage node i
$S^{(s,i)}$	The secondary sliver held by storage node i
$\{S^{(p,*)}\}_{f+1}$	Any set of $f + 1$ primary slivers
$S_0^{(x,i)}$	The source symbols of the sliver $S^{(x,i)}$
$\bar{x}$	The sliver type other than x , i.e., $\bar{p} = s$
t_x	Reconstruction threshold: $t_p = 2f + 1, t_s = f + 1$
$p_x(i, j)$	Position of the symbol shared by $S^{(x,i)}$ and $S^{(\bar{x},j)}$
M	The blob metadata (M^p, M^s)
M^p	Metadata associated with the primary slivers
M^s	Metadata associated with the secondary slivers
h	The blob commitment over the metadata M
id	The blob ID, i.e., $\text{HASH}(h, \text{size})$

Lemma 2 (Secondary Sliver Reconstruction). *If a party holds a set of $(f + 1)$ symbols $\{E_{(*,i)}\}_{f+1}$ from a secondary sliver $S^{(s,i)}$, it can obtain the complete secondary sliver $S^{(s,i)}$.*

Algorithm 3 WALRUS store operations.

```

1: N                                ▷ identifier of the storage node
2: dbm                             ▷ persists this node's encoded metadata part
3: dbb                             ▷ persists slivers

// Store slivers
4: procedure STORESLIVERS(StoreRqst)
5:   (id, size, M, S(p,i), S(s,i)) ← StoreRqst    ▷ i is this node's index
6:   if ¬ISREGISTERED(id) then return ⊥
7:   id' ← MAKEBLOBID(M, size)
8:   if id' ≠ id then return ⊥
9:   if ¬VERIFYSLIVER(S(p,i), M) then return ⊥
10:  if ¬VERIFYSLIVER(S(s,i), M) then return ⊥
11:  (Mp, Ms) ← M
12:  dbm[id] ← (ENCODEMETADATA(M)i,
              OPENING(Mp, i), OPENING(Ms, i))
              ▷ metadata piece and openings of the blob commitment
13:  dbb[id] ← (S(p,i), S(s,i))    ▷ may be truncated (section 3.3)
14:  return SIGN(id)              ▷ signed acknowledgment

// Serve slivers (primary path for reads)
15: procedure SERVESLIVERS(SliversRqst)
16:   id ← SliversRqst
17:   if ¬READCERTIFICATE(id) then
18:     return ⊥    ▷ ensure on-chain availability certificate exists
19:   (S(p,i), S(s,i)) ← dbb[id]
20:   return S(p,i)    ▷ serve primary sliver for read threshold f + 1

// Recover one sliver of this node from verified intersections
21: procedure RECOVERSLIVER(id, nodes, i, M, x)    ▷ sliver type x ∈ {p, s}
    // ask each node m with index j for the symbol shared by
    // S(x,i) and S(x,j), opened against Mjx
22:   Q ← awaitix : {(j, z, π) ← REQUESTSYMBOL(m, id, px(i, j)) :
                  m ∈ nodes, j = NODEINDEX(m, nodes),
                  VERIFYOPENING(Mjx, z, π, i) }
23:   X ← {(j, z) : (j, z, π) ∈ Q}    ▷ verified symbols
24:   S0(x,i) ← ERASUREDECODE(X, tx, n)    ▷ source symbols
25:   S(x,i) ← ERASUREENCODE(S0(x,i), tx, n)    ▷ fully extended sliver
26:   if MERKLETREE(S(x,i)) ≠ Mix then
27:     return (⊥, INCONSISTENCYPROOF(Q))    ▷ see section 4.4
28:   return (S(x,i), ⊥)

// Recover slivers for this node (symbol-level)
29: procedure RECOVERSLIVERS(id, nodes)
30:   if ¬READCERTIFICATE(id) then return ⊥    ▷ only past the PoA
31:   M ← RETRIEVEMETADATA(id)
32:   ▷ required to verify openings; defined in algorithm 2
33:   (Mp, Ms) ← M
34:   i ← NODEINDEX(N, nodes)    ▷ this node's row/column index
35:   (S(s,i), φ) ← RECOVERSLIVER(id, nodes, i, M, s)    ▷ secondary first
36:   if φ ≠ ⊥ then return φ    ▷ inconsistency proof
37:   (S(p,i), φ) ← RECOVERSLIVER(id, nodes, i, M, p)
38:   if φ ≠ ⊥ then return φ    ▷ inconsistency proof
39:   dbb[id] ← (S(p,i), S(s,i))    ▷ persist both slivers
40:   dbm[id] ← (ENCODEMETADATA(M)i,
              OPENING(Mp, i), OPENING(Ms, i))
41:   return ok

```

PROOF. The proof directly follows from the reconstruction property of erasure codes with reconstruction threshold $(f + 1)$. $\square$

Theorem 1. *RED STUFF satisfies Write Completeness (definition 1).*

PROOF. To write a blob B , an honest writer W sends at least $(2f + 1)$ correctly encoded slivers (parts) to different storage nodes, along with a binding vector commitment M over those slivers. For these nodes the property holds by definition. Now let us assume a node i that is not in the initial $2f + 1$ recipients. The node will ask every node j for the shared symbols in its primary (i.e., $E_{(i,j)}$) and secondary (i.e., $E_{(j,i)}$) sliver. Given the binding vector commitment M , node j can either send the true symbols or not reply. Since at least $2f + 1$ nodes acknowledged M , node i will get $f + 1$ correct symbols for its primary sliver $\{E_{(i,*)}\}_{f+1}$ and $f + 1$ correct symbols for its secondary sliver $\{E_{(*,i)}\}_{f+1}$. From lemma 2, this means that i will reconstruct its full secondary sliver $S^{(s,i)}$.

Since this reasoning applies to any generic node i , it holds for all nodes. As a result, eventually all $2f + 1$ honest nodes will reconstruct their secondary slivers $S^{(s,*)}$. Every time a node j reconstructs its secondary sliver, it also replies to node i with the shared symbol, which is part of the primary sliver of i (i.e., $E_{(i,j)}$). As a result, eventually i will go from $\{E_{(i,*)}\}_{f+1}$ to $\{E_{(i,*)}\}_{2f+1}$. This allows node i to apply lemma 1 and reconstruct its primary sliver $S^{(p,i)}$.

Since this reasoning applies to any node i , it holds for all nodes and concludes the proof that all honest nodes will eventually hold both their primary and secondary slivers. $\square$

6.2 Read Consistency

We prove that RED STUFF satisfies Read Consistency. Informally, if two honest readers read a blob B stored on the network, they either both eventually obtain B or both eventually fail and obtain $\perp$.

Theorem 2. *RED STUFF satisfies Read Consistency (definition 1).*

PROOF. Note that the encoding scheme is deterministic and the last step of reading is to re-run the encoding and reconstruct M . As a result, a reader that accepts the read as correct needs to output B .

The challenge with Read Consistency is whether the writer can convince two readers that collect different slivers to output B and $\perp$, respectively. Let us assume that two honest readers R_1 and R_2 read a blob B from the network and R_1 eventually obtains B while R_2 eventually obtains $\perp$.

There are two scenarios for R_2 to output $\perp$:

- (1) R_2 gets $f + 1$ replies that match the vector commitment M and tries to reconstruct. After reconstruction, re-encoding, and recomputing the vector commitment, it does not match M .
- (2) Some node failed to reconstruct its secondary sliver. By the algorithm, this node will hold a proof of inconsistency, which it will send to R_2 .

In either scenario, R_1 should also have detected the inconsistency during the reconstruction and output $\perp$; otherwise, the binding property of the vector commitment would not hold. Hence a contradiction. $\square$

6.3 Validity

We prove that RED STUFF satisfies Validity. Informally, if an honest writer writes a correctly encoded blob B to the network, every honest reader eventually obtains B .

Theorem 3. *RED STUFF satisfies Validity (definition 1).*

PROOF. To write a blob B , an honest writer W constructs n correctly encoded slivers (parts) along with a binding vector commitment M over those slivers. Since the writer is honest, from theorem 1, at least $2f + 1$ honest storage nodes will hold their respective slivers. Let nodes denote the entire set of storage nodes. An honest reader queries each storage node $N \in \text{nodes}$ for its primary slivers, verifies each against M , and, once it holds $f + 1$, uses them to reconstruct B . Since all honest storage nodes will eventually reply to the reader and W was honest, the reader will eventually obtain B . $\square$

7 Evaluation

Our evaluation aims at demonstrating the following claims:

- C1** WALRUS achieves *low latency*, bounded by network delay.
- C2** WALRUS clients achieve *high read and write throughput*.
- C3** WALRUS *reconfigures and recovers* without downtime.

To make the evaluation as realistic as possible, we perform measurements on the public WALRUS deployments described below, thus exposing the system to real-world conditions, real users, and infrastructure outside our control.

7.1 Production Deployments

The WALRUS protocol has powered a decentralized production testnet since October 2024 (redployed in April 2025), and a mainnet since March 2025. The storage nodes and clients on these networks are written in asynchronous multi-core Rust. They use axum [25] for HTTPS networking, fastcrypto [14] for cryptography, rocksdb [13] for storage, reed-solomon-simd [16] for erasure coding, and Sui [23] as a fast blockchain. The code is open-source and available at <https://github.com/MystenLabs/walrus>.

Both networks use the same encoding parameters with 1000 shards across approximately 100 storage nodes, although the number of nodes and their shard distribution varied slightly over time. The following values were observed on 2026-07-29:⁶

Mainnet. The WALRUS mainnet comprises 95 storage nodes in 17 countries, the top 5 in terms of the total stake being the USA, Finland, the Netherlands, Germany, and Lithuania. No single country has more than 1/3 of the stake. Besides Europe and North America, nodes are also located in the APAC and LATAM regions. Nodes hold between 4 and 26 shards (median: 7, P90: 22).

Most storage nodes run a recent LTS version of Ubuntu with at least 16 CPU cores, 128 GB RAM, and 1 Gbps bandwidth. Hardware varies across Intel and AMD CPUs and HDD, SSD, and NVMe storage. Nodes offer between 7.0 TB and 2.03 PB of storage (median: 55 TB, P90: 100 TB, P99: 362 TB). The total system capacity (encoded

⁶Committee, shard, and capacity values are read from the on-chain state. Blob counts, registered and active sizes, and peak-day figures are based on our index of WALRUS events on Sui. Country distribution is based on geolocation of the nodes' public IP addresses. Hardware information was self-reported by operators in a questionnaire. Some of these values are also shown on <https://walrus.chainviz.io>.

Table 3: Time to write and read on Filecoin [8] and to store until finality on Arweave for 10 kB, 10 MB, 20 MB, 70 MB, and 135 MB files. Arweave reads are shown in figure 5.

System	Operation	Latency
Filecoin	Write & secure seal	1.5 h
	Read from secured copy	3 h
Arweave	Store to finality (10 kB)	38 min
	Store to finality (10 MB–135 MB)	35–36 min

size) is 4.00 PB, 56% of which is currently used.⁷ Aside from the large storage capacity, nodes are modestly provisioned similarly to typical quorum-based blockchains [24].

Since the public launch in March 2025, 17.6 million unique blobs have been registered by 9,675 wallets with a total unencoded size of 686 TB, 477 TB of which is still active. The maximum number of blobs certified in one day was around 834,000 on 2025-04-02 (approximately 10 blobs per second). The maximum total unencoded size of blobs certified in one day was 21.0 TB on 2026-01-29 (corresponding to an average goodput of approximately 1.95 Gbps).

Testnet. The WALRUS testnet comprises 101 storage nodes with a geographical and stake distribution similar to that of the mainnet. Node specifications are slightly lower, with most nodes having at least 8 CPU cores and 64 GB RAM. Storage capacities vary between 322 GB and 75 TB (median: 5.0 TB, P90: 16.5 TB), resulting in a system capacity of 333 TB.

7.2 Baselines

The other two main operational decentralized storage networks are Filecoin and Arweave. To compare WALRUS with Filecoin, we focus on two metrics: (1) the time until a client obtains an incentivized cryptographic guarantee that a file has been durably stored, and (2) the time to read the sealed (secured) copy, as opposed to a transient cached “hot” copy. Because Filecoin relies on resource-intensive proofs of storage [9], the latency to reach these guarantees is (within sector limits, up to 32 GiB) largely independent of file size. Table 3 reports these latencies: 1.5 hours to write and seal, and 3 hours to read the sealed copy [8].

Arweave guarantees storage through its mining algorithm, which takes tens of minutes and allows it to quickly fulfill storage promises when compared to Filecoin. Table 3 illustrates the end-to-end latency to store blobs of varying sizes on Arweave. Write latency averages 35 to 38 minutes, regardless of the blob's size, highlighting that the bottleneck is in the mining algorithm.

The following sections demonstrate that WALRUS's separation of consensus from data storage results in latencies that are orders of magnitude smaller than those of both Filecoin and Arweave.

7.3 System Performance

We evaluated WALRUS's performance on the *public testnet* by collecting latency and throughput measurements as of 2026-01-12

⁷The “system capacity” here refers to the contract-enforced capacity based on a vote by the committee. The total amount of storage reported by nodes in the committee is 7.86 PB.

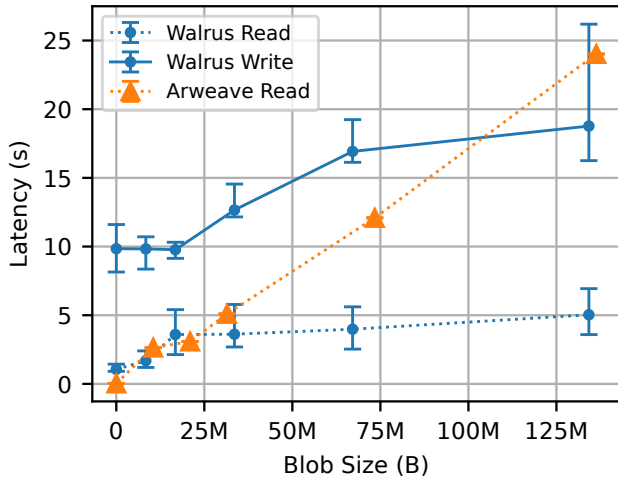


Figure 5: Median read and write latencies for various unencoded blob sizes on WALRUS and read latencies on Arweave. Arweave writes require minutes and so are presented in table 3. Error bars indicate the 5th and 95th percentile latencies.

from an AWS m5d.8xlarge instance (10 Gbps bandwidth, 32 vCPUs, 128 GB RAM) located in US East (N. Virginia). This client was used for evaluation and is not part of the WALRUS testnet.

WALRUS Latency. Figure 5 illustrates the end-to-end latencies of WALRUS for 50 stores and reads of random blobs of various sizes. Up to four blobs were written or read concurrently; all blobs were written before any were read. The measurements span from before the client encodes the blob until it creates a PoA on the blockchain, which is after finality but may be before all 1000 shards have received their respective slivers.

The read latencies of both WALRUS and Arweave were low, even for large blobs. On WALRUS, blobs up to 135 MB were read in less than 5 seconds. By contrast, Arweave’s read latencies were around 25 seconds for the 135 MB files. This difference follows from how the two systems retrieve data: a WALRUS client requests $f + 1$ primary slivers concurrently from several different storage nodes (algorithm 2), each carrying roughly $1/(f + 1)$ of the blob, so the download aggregates the bandwidth of the nodes it contacts and is ultimately limited by the client’s own link. An Arweave client instead downloads the entire file from a single node, whose resources are also shared with the consensus protocol.

Median write latencies on WALRUS were higher than read latencies and remained less than 20 seconds for blobs up to 135 MB. For small blobs, less than 20 MB, the write latencies were consistently around 10 seconds. Both were significantly lower than Arweave’s write to finality, which required over 30 minutes (table 3).

Figure 6 provides more details on WALRUS’s write latencies for the various blob sizes. Each write operation consists of five key steps: *encoding* (erasure encoding the blob), *check status* (checking prior storage status of the blob), *register* (reserve space for the blob), *data upload* (uploading slivers to storage nodes and retrieving the signed acknowledgments that form the availability certificate), and *publish PoA* (committing the PoA to the blockchain).

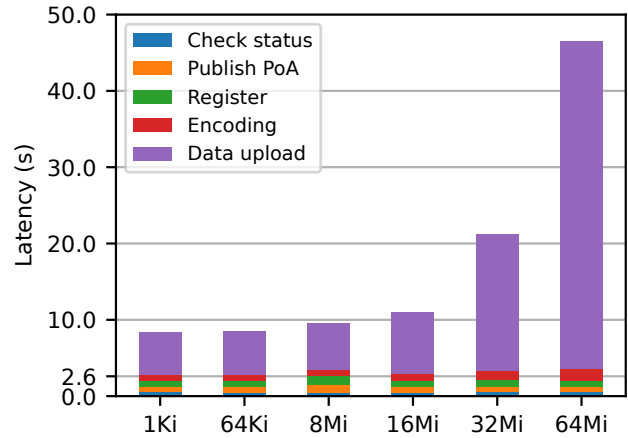


Figure 6: Latencies for various operations during a single blob upload for various sizes. The data upload latency indicates the latency to upload to all online shards, as opposed to the minimum necessary quorum.

These operations either have constant latencies: check status, register, publish PoA; or variable latencies: encoding and data upload. For these six blob sizes, the constant latencies were less than 2.6 seconds. In contrast, most of the write time was dominated by the uploads of the slivers and metadata and the retrieval of the acknowledgments. For small blobs, this upload latency was primarily composed of the overhead for the constant-sized metadata uploads, but grew linearly from around 8 MiB upward. Therefore, these results validate our claim **C1**: WALRUS achieves low latency and is bounded by network delays.

Client Throughput. Figure 7 illustrates the throughput (in bytes per second) that a single WALRUS and Arweave client can achieve. The read throughput of WALRUS was measured by requesting stored blobs at a constant rate for 10–20 seconds (depending on the blob size) before increasing the request rate by 30–60 requests/minute and repeating. The throughput was then the highest constant request rate before the first read failure (e.g., timeout). The write throughput was measured by performing uploads with a constant 8 concurrent requests, the client’s configured concurrency, which was taken as a reasonable default for a commodity system. In both cases, the reported throughput is the request throughput multiplied by the blob size. Arweave’s write throughput is omitted: its client communicates only with a central gateway, so we could not measure it reliably.

As expected, WALRUS’s read throughput increases with the blob size, as fewer requests for larger amounts of data better utilize the network connections between the client and the storage nodes. Accordingly, the single client achieved around 400 MB/s throughput for 135 MB blobs. By contrast, Arweave’s read throughput was severely limited by the blockchain interactions. The WALRUS client’s write throughput linearly increased to around 62 MB/s. The limiting factor here was the client’s configuration of 8 workers and concurrent requests. This could be relaxed until the bottleneck becomes the user’s CPU or network, as each worker maintains its own blockchain state and Sui supports a much higher throughput

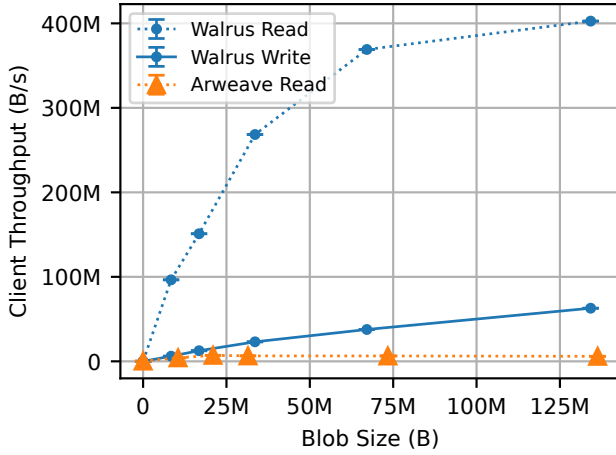


Figure 7: Single-client throughput of WALRUS and Arweave for various unencoded blob sizes.

in transactions per second. However, this configuration is already sufficient to saturate many residential connections. In contrast, Arweave’s client plateaued at around 7 MB/s, which is the limit of the consensus protocol to agree on storing data. As such, these results validate claim C2: WALRUS enables clients to read and write at high throughput.

7.4 Epoch-Change Overhead

The main overhead associated with the transition from one WALRUS epoch to another arises from shards being relocated across epochs, that is, when a shard assigned to a node in epoch e is reassigned to a different node in epoch $e + 1$.

The transfer of a shard to another node requires sending slivers of all active blobs and, if the new owner was not part of the committee in epoch e , all blob metadata. During a *shard transfer*, the previous owner of the shard sends the shard’s slivers to the new owner. If the previous owner is unwilling or unable to do so, the receiving node instead performs *shard recovery*.

Shard reassignments on mainnet have been frequent but mostly small. Across the 36 epochs between the launch in March 2025 and July 2026, twelve epoch changes reassigned five or more shards: epochs 2, 6, 9, 11, 16, 20, 23, 27, 28, 30, 32, and 36. The largest reassignment moved 27 shards at epoch 2 (2025-04-08), when the network was young and held 4.4 million blobs; the largest since then moved 13 shards, at epochs 28 (2026-04-07) and 36 (2026-07-28). In at least three cases, shards were moved from a node that was no longer available, requiring the new owners to *recover* their new shards (epochs 17, 20, and 23). We describe two *shard transfers* and two *shard recoveries* in further detail.⁸

At epoch 9 (2025-07-15), a joining node received 4 shards. For the 9.9 million blobs that were live at the time, a total of 636 GB of blob metadata was transferred over 12.5 hours at around 113 Mbps; and 79.5 million slivers, 890 GB, over 15.25 hours at around 129 Mbps.

⁸Committee memberships and shard assignments are computed from the system object at each epoch. Sliver counts, transfer volumes, and durations are based on Prometheus metrics voluntarily reported by node operators, which is also how we distinguish transferred from recovered shards.

At epoch 11 (2025-08-12), a node holding 12 shards left the committee and its shards were reassigned to twelve different nodes, one each. Every receiving node was already in the previous committee and therefore held the metadata, so only slivers had to be transferred: 6.6 million primary and 6.6 million secondary slivers per shard, about 210 GB each and 2.5 TB in total. Since each receiver synchronized independently of the others, up to seven shards were transferred concurrently, at an aggregate rate of roughly 1.4 Gbps on average and 2.5 Gbps at peak, and the last shard completed about four hours after the epoch change.

We also analyze two epoch changes that required recovery because the node leaving the committee was no longer online to hand over its slivers. At epoch 17 (2025-11-04), a node holding 4 shards was removed, and each of its shards held slivers for approximately 465,000 blobs.⁹ Three of these shards were reassigned to a single node, which recovered them sequentially in 17 to 21 hours each, and the fourth was recovered by another node. At epoch 20 (2025-12-16), a node holding 7 shards was removed and its shards were distributed across seven nodes, each recovering slivers for approximately 521,000 blobs. Five shards were recovered concurrently, which took approximately 64 hours in total. Recovery of one of the other shards took 16 hours; the seventh recovery was interrupted and resumed only after its node came back online.

These cases demonstrate that WALRUS can efficiently recover even from the complete failure of a node holding several shards. Note that in all of these cases, WALRUS remained available for storing and reading data, thus validating claim C3.

8 Related Work

IPFS [2] provides a decentralized store for files, and is extensively used by blockchain systems and decentralized apps for their storage needs. It provides content-addressable storage for blocks, and uses a distributed hash table (DHT) to maintain a link between file replicas and nodes that store them. Publishers must pin files to storage nodes, usually for a fee, to ensure the files remain available. The underlying storage uses replication on a few nodes for each file.

Filecoin [17] extends IPFS, using a longest-chain blockchain and a cryptocurrency (FIL) to incentivize storage nodes. Publishers acquire storage contracts with a few nodes, and payments are made in the cryptocurrency. Filecoin mitigates the risk that these nodes delete the replicas by requiring storage nodes to hold differently encoded copies of the file, and performing challenges against each other for the encoded files. These copies are encoded in such a way that it is slow to reproduce them from the original copy, to avoid relay attacks. As a result, if the user wants to access the original file, they need to wait a long time for the decoding of a copy, unless some storage node has a hot copy. Since there is no in-built incentive for storing hot copies, this service usually costs extra.

Arweave [27] mitigates slow reads through a Proof-of-Access algorithm that incentivizes storage nodes to store as many files as possible to maximize rewards. This is implemented in conjunction with a full replication strategy, and results in replication levels almost equal to those of classic state machine replication. Additionally, the system only allows files to be stored “forever”, through

⁹The drop in the number of blobs relative to earlier epochs is due to the fact that users started bundling multiple files within a single blob around June 2025 to reduce overheads and cost, and earlier blobs expired over time.

a mechanism of pre-payment—which lacks the flexibility to control lifetime and deletion, and is capital-inefficient since payment is upfront.

In contrast to Filecoin and Arweave, WALRUS uses erasure coding to maintain a very low overhead of $4.5\times$ while ensuring that data survives the loss of up to $2/3$ of the shards and that the system continues to accept writes even if up to $1/3$ of the shards are unresponsive. Furthermore, WALRUS does not implement its own separate blockchain for node management but uses Sui instead.

Storj [21] is a distributed storage solution that leverages encoding to achieve a low replication factor. Its deployed network implements a Reed-Solomon-based erasure-coding scheme with a $29/80$ configuration [22], wherein a file is encoded into 80 parts, with any 29 sufficient for reconstruction. This approach results in a $2.75\times$ replication factor, offering a substantial reduction in storage costs compared to prior systems. However, a key limitation is its inability to efficiently heal lost parts. The system relies on users to reconstruct the full file and subsequently re-encode it to facilitate the recovery of lost parts. In contrast, WALRUS's use of RED STUFF incorporates an efficient reconstruction mechanism, which is critical for the healing property of the erasure-coding scheme, especially under the churn that occurs naturally in a permissionless system. RED STUFF builds on the Twin-code framework [11, 18], which uses two linear encodings of data to enhance the efficiency of sliver recovery. However, unlike Twin-code, RED STUFF encodes data across differently sized dimensions and integrates authenticated data structures, achieving Write Completeness (as defined in definition 1) and ensuring Byzantine fault tolerance.

Modern blockchains provide some storage, but it is prohibitively expensive to store larger blobs due to the costs of full replication across all validators, as well as potentially long retention times to allow verifiability. Within the Ethereum ecosystem specifically, the current scaling strategy around L2s involves posting blobs on the main chain that represent bundles of transactions to be executed and verified via either zero-knowledge or fraud proofs. Specialized networks, such as Celestia, based on availability sampling [1], have emerged to fulfill this need off the main Ethereum chain. In Celestia, two-dimensional Reed-Solomon codes are used to encode blobs, and symbols are distributed to light nodes to support “trustless” availability. However, all blobs are fully replicated across the validators of the system for a limited time period of about a month. WALRUS instead offers proofs of availability with arbitrarily long retention periods and a reduced cost of storage per node, which allows the system to scale inexpensively.

The most closely related work to ours is Semi-AVID [15], which has also been explored as an alternative to provide data availability for rollups. It is similar to the Strawman II design, meaning that although it can achieve the critical property of verifiable data storage, it cannot achieve Write Completeness unless the full data is reconstructed. This makes it prohibitively expensive for epoch change and only suitable for either permissioned systems with no churn or short-lived data storage. This is the main challenge with AVID [5], which is optimized to provide verifiability of data storage, disallowing the output of $\perp$. This is overkill, as a malicious writer can simply encode garbage data instead of a failed encoding. Hence, the machinery to detect and reject failed encodings that AVID provides is unnecessarily constraining and expensive.

9 Conclusion

We introduce WALRUS, a novel approach to decentralized blob storage that leverages fast erasure codes and modern blockchain technology. By utilizing the RED STUFF encoding algorithm and the Sui blockchain, WALRUS achieves high resilience and low storage overhead while ensuring efficient data management and scalability. Our system operates in epochs, with all operations sharded by *id*, enabling it to handle large volumes of data. The innovative two-dimensional BFT encoding protocol of RED STUFF allows for efficient data recovery, load balancing, and dynamic availability of storage nodes, addressing key challenges faced by existing decentralized storage systems.

Furthermore, WALRUS provides a committee reconfiguration protocol that guarantees uninterrupted data availability during network evolution. Our contributions include defining the problem of Asynchronous Complete Data Storage, presenting the RED STUFF protocol, and providing a full end-to-end system design with WALRUS, including a production-grade implementation and deployment, paving the way for future advancements in decentralized storage technologies.

Generative AI Usage

The authors used Claude, via the Cursor IDE and Claude Code, to assist with copy-editing (spelling, grammar, punctuation, and notational consistency), verification of the bibliography against Crossref and DBLP, and LaTeX formatting. All AI-assisted changes were reviewed and approved by the authors. All content of the paper was written by the authors themselves, who retain full responsibility for the accuracy, originality, and integrity of the work.

Acknowledgments

This work was funded by Mysten Labs. We would like to express our gratitude to Dmitri Perelman, He Liu, Pei Deng, Sadhan Sood, William Bradley, and Zhe Wu for their invaluable contributions to bringing WALRUS to production and to Damir Shamanev for his assistance in constructing the smart contracts that connect WALRUS with the Sui blockchain. We extend a special thank you to Joachim Neu for identifying a serious vulnerability in our previous testnet implementation. This vulnerability was related to its use of RaptorQ for erasure coding and led us to replace RaptorQ with RS codes.

References

- [1] Mustafa Al-Bassam, Alberto Sonnino, Vitalik Buterin, and Ismail Khoffi. 2021. Fraud and data availability proofs: Detecting invalid blocks in light clients. In *Financial Cryptography and Data Security (FC)*. Springer, Berlin, Heidelberg, 279–298. doi:10.1007/978-3-662-64331-0_15
- [2] Juan Benet. 2014. IPFS - Content addressed, versioned, P2P file system. arXiv:1407.3561 [cs.NI] doi:10.48550/arXiv.1407.3561
- [3] Nazanin Zahed Benisi, Mehdi Aminian, and Bahman Javadi. 2020. Blockchain-based decentralized storage networks: A survey. *Journal of Network and Computer Applications* 162 (2020), 102656. doi:10.1016/j.jnca.2020.102656
- [4] Sam Blackshear, Andrey Chursin, George Danezis, Anastasios Kichidis, Lefteris Kokoris-Kogias, Xun Li, Mark Logan, Ashok Menon, Todd Nowacki, Alberto Sonnino, Brandon Williams, and Lu Zhang. 2024. Sui Lutris: A blockchain combining broadcast and consensus. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, New York, NY, USA, 2606–2620. doi:10.1145/3658644.3670286
- [5] Christian Cachin and Stefano Tessaro. 2005. Asynchronous verifiable information dispersal. In *IEEE Symposium on Reliable Distributed Systems (SRDS)*. IEEE Computer Society, Piscataway, NJ, USA, 191–202. doi:10.1109/RELDIS.2005.9

- [6] Dario Catalano and Dario Fiore. 2013. Vector commitments and their applications. In *Public-Key Cryptography (PKC)*. Springer, Berlin, Heidelberg, 55–72. doi:10.1007/978-3-642-36362-7_5
- [7] John R. Douceur. 2002. The Sybil attack. In *International Workshop on Peer-to-Peer Systems (IPTPS)*. Springer, Berlin, Heidelberg, 251–260. doi:10.1007/3-540-45748-8_24
- [8] Filecoin. 2026. Network performance. Retrieved July 21, 2026 from <https://docs.filecoin.io/networks-and-tools/networks/mainnet/network-performance>
- [9] Ben Fisch. 2018. PoReps: Proofs of space on useful data. <https://eprint.iacr.org/2018/678> Cryptology ePrint Archive, Paper 2018/678.
- [10] Chuanlei Li, Minghui Xu, Jiahao Zhang, Hechuan Guo, and Xiuzhen Cheng. 2024. SoK: Decentralized storage network. *High-Confidence Computing* 4, 3 (2024), 100239. doi:10.1016/j.hcc.2024.100239
- [11] Ninoslav Marina, Aneta Velkoska, Natasha Paunkoska, and Ljupcho Baleski. 2015. Security in Twin-code framework. In *International Congress on Ultra Modern Telecommunications and Control Systems and Workshops (ICUMT)*. IEEE, Piscataway, NJ, USA, 247–252. doi:10.1109/ICUMT.2015.7382437
- [12] Ralph C. Merkle. 1988. A digital signature based on a conventional encryption function. In *Advances in Cryptology (CRYPTO)*. Springer, Berlin, Heidelberg, 369–378. doi:10.1007/3-540-48184-2_32
- [13] Meta Platforms. 2012. RocksDB. Retrieved July 21, 2026 from <https://rocksdb.org>
- [14] Mysten Labs. 2022. fastcrypto. Retrieved July 21, 2026 from <https://github.com/MystenLabs/fastcrypto>
- [15] Kamilla Nazirkhanova, Joachim Neu, and David Tse. 2022. Information dispersal with provable retrievability for rollups. In *ACM Conference on Advances in Financial Technologies (AFT)*. ACM, New York, NY, USA, 180–197. doi:10.1145/3558535.3559778
- [16] Anders Trier Olesen. 2023. reed-solomon-simd. Retrieved July 21, 2026 from <https://github.com/AndersTrier/reed-solomon-simd>
- [17] Yiannis Psaras and David Dias. 2020. The InterPlanetary File System and the Filecoin network. In *IEEE/IFIP International Conference on Dependable Systems and Networks, Supplemental Volume (DSN-S)*. IEEE, Piscataway, NJ, USA, 80–80. doi:10.1109/DSN-S50200.2020.00043
- [18] K. V. Rashmi, Nihar B. Shah, and P. Vijay Kumar. 2011. Enabling node repair in any erasure code for distributed storage. In *IEEE International Symposium on Information Theory (ISIT)*. IEEE, Piscataway, NJ, USA, 1235–1239. doi:10.1109/ISIT.2011.6033732
- [19] Irving S. Reed and Gustave Solomon. 1960. Polynomial codes over certain finite fields. *J. Soc. Indust. Appl. Math.* 8, 2 (1960), 300–304. doi:10.1137/0108018
- [20] Fred B. Schneider. 1990. Implementing fault-tolerant services using the state machine approach: a tutorial. *Comput. Surveys* 22, 4 (1990), 299–319. doi:10.1145/98163.98167
- [21] Storj Labs, Inc. 2018. Storj: A decentralized cloud storage network framework. <https://www.storj.io/storjv3.pdf>
- [22] Storj Labs, Inc. 2026. Understanding file repair in a distributed network. Retrieved July 30, 2026 from <https://storj.dev/learn/concepts/file-repair>
- [23] Sui Foundation. 2023. Sui. Retrieved July 21, 2026 from <https://sui.io>
- [24] Sui Foundation. 2026. Validator deployment and configuration. Retrieved July 21, 2026 from <https://docs.sui.io/guides/operator/validator/validator-config>
- [25] Tokio. 2021. axum. Retrieved July 21, 2026 from <https://github.com/tokio-rs/axum>
- [26] David Vorick and Luke Champine. 2014. Sia: Simple decentralized storage. Whitepaper, Nebulous Inc. <https://sia.tech/sia.pdf>
- [27] Sam Williams, Viktor Diordiiev, Lev Berman, India Raybould, and Ivan Uemlianin. 2019. Arweave: A protocol for economically sustainable information permanence. Arweave yellow paper, DRAFT-1. <https://www.arweave.org/yellow-paper.pdf>
- [28] Ennan Zhai, Ruichuan Chen, David Isaac Wolinsky, and Bryan Ford. 2014. Heading off correlated failures through Independence-as-a-Service. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, Berkeley, CA, USA, 317–334. <https://www.usenix.org/system/files/conference/osdi14/osdi14-paper-zhai.pdf>

A Open Science

A copy of the code and configuration artifacts related to this article is available at <https://doi.org/10.5281/zenodo.21720661>. It includes the following items:

- (1) WALRUS storage node source code,
- (2) WALRUS client source code,
- (3) the code used to perform the evaluations on WALRUS,
- (4) instructions on how to perform the evaluation,
- (5) client configuration for WALRUS testnet.

Information on the code structure and instructions to utilize the above artifacts can be found in the top-level README.md file of the repository.

The artifact is configured with an existing Sui wallet; however, as the code is public, parallel use of the configured wallet will likely result in it becoming locked on Sui. We therefore recommend generating a new Sui wallet address and funding the wallet with free testnet Sui.

Finally, since the evaluation was performed on the live WALRUS testnet, we note that both the static code above and the measurements may become incompatible or de-synchronize with the network as it continues to evolve and improve.